
Software Requirements Specification

for

PrepVault

Version 1.0

Prepared by Praveenkumar B

Dept of CSE , CIT

03-07-2025

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Project Scope	2
1.5 References.....	2
2. Overall Description	3
2.1 Product Perspective.....	3
2.2 Product Features	3
2.3 User Classes and Characteristics	4
2.4 Operating Environment.....	4
2.5 Design and Implementation Constraints	4
2.6 User Documentation	5
2.7 Assumptions and Dependencies	5
3. System Features	6
3.1 System Feature 1	6
3.2 System Feature 2.....	6
3.3 System Feature 3	7
3.4 System Feature 4.....	7
3.5 System Feature 5.....	7
3.6 System Feature 6.....	8
4. External Interface Requirements	9
4.1 User Interfaces	9
4.2 Hardware Interfaces	9
4.3 Software Interfaces	10
4.4 Communications Interfaces	10
5. Other Nonfunctional Requirements.....	11
5.1 Performance Requirements	11
5.2 Safety Requirements	11
5.3 Security Requirements	11
5.4 Software Quality Attributes	11
6. Other Requirements	12
Appendix A: Glossary.....	13
Appendix B: Analysis Models.....	14
Appendix C: Source Code	26
Appendix D: Output Screenshots	34

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This SRS describes the requirements for **PrepVault**, a web-based platform designed to help users **organize and track their DSA problem-solving progress** through user-defined sheets, without AI-based code assistance or execution. It focuses on managing problem-solving data for interviews, including tracking, tagging, revision planning, and analytics.

1.2 Document Conventions

This SRS follows IEEE formatting guidelines and uses standard typographical styles to denote different elements within the document. Section headings are numbered hierarchically (e.g., 1.1, 1.2) for clarity and traceability. Functional requirements are labeled using the format REQ-x.x (e.g., REQ-3.2) to uniquely identify and reference them throughout the documentation. Boldface text is used to highlight key terms and modules, while inline code or tag values (e.g., Solved, Review) are shown using monospace formatting. All requirements in this document are considered to have equal priority unless explicitly stated otherwise..

1.3 Intended Audience and Reading Suggestions

Role	Purpose of Reading
End Users	To understand how PrepVault supports DSA interview preparation, track progress, and receive personalized suggestions.
Admin	To understand system deployment, data management, maintenance requirements, and overall operational responsibilities.

1.4 Project Scope

PrepVault is a web-based DSA interview preparation tracker that aims to provide an intuitive, distraction-free environment for aspiring software engineers to manage and track their problem-solving journey. The platform enables users to create topic-wise or company-specific problem sheets, mark problems with statuses such as Solved, Tried, or Review, and receive rule-based suggestions based on their topic-wise progress and difficulty distribution. Additionally, it provides visual analytics in the form of charts to help users identify strong and weak areas, plan revisions, and maintain consistency with features like streak tracking and daily goals. This version is the first release intended for academic evaluation and future enhancement.

1.5 References

- IEEE Standard 830-1998: Recommended Practice for Software Requirements Specifications
- Firebase Documentation – <https://firebase.com/docs>
- ReactJS Official Docs – <https://reactjs.org/docs/getting-started.html>
- Tailwind CSS Docs – <https://tailwindcss.com/docs>
- Chart.js Documentation – <https://www.chartjs.org/docs/latest/>

2. Overall Description

2.1 Product Perspective

PrepVault is a self-contained, web-based application developed from scratch for the purpose of structured DSA interview preparation. It is not part of a larger system and does not serve as a plug-in or module for any third-party product. The application is entirely browser-based and utilizes open-source technologies such as React.js for the frontend and Supabase for backend services including authentication and PostgreSQL-based data storage. The product follows a modular architecture with distinct modules for sheet management, analytics, progress tracking, and suggestion generation. All components operate within a single cohesive interface designed for individual users preparing for software engineering interviews..

2.2 Product Features

The major features of **PrepVault** include:

- **Sheet Creation and Management:** Users can create multiple sheets for different preparation goals (e.g., "Amazon SDE Prep", "DP Mastery"), add problems manually with metadata such as title, platform, tags, difficulty, and current status.
- **Problem Status Tracking:** Each problem can be tagged with statuses like Not Tried, Tried, Solved, or Review, allowing the user to maintain a clear view of their preparation progress.
- **Rule-Based Problem Suggestion:** The system provides intelligent problem suggestions using predefined rules that consider topic completion, problem status, and difficulty gaps.
- **Analytics Dashboard:** Visual charts display topic-wise strengths and weaknesses, problem distribution by difficulty and tag, and completion statistics across sheets.
- **Revision Planner:** Problems marked for review are aggregated and prioritized for revision based on time gaps, previous attempts, and topic balance.

2.3 User Classes and Characteristics

There are three primary user types for the platform:

- **Standard Users:** These are students or professionals preparing for coding interviews. They typically have intermediate knowledge of DSA and require organizational tools to streamline their prep.
- **Power Users:** These users rely heavily on detailed analytics and revision tracking. They tend to manage multiple sheets and actively analyze their weak areas.
- **Project Evaluators (Faculty):** As this is a lab project, evaluators or faculty members reviewing the application will access the system mainly for demonstration and assessment of functional correctness, usability, and technical soundness.

2.4 Operating Environment

The application is built as a responsive Single Page Application (SPA) using the following technologies:

- **Frontend:** React.js with Tailwind CSS for UI components and styling
- **Backend:** Firebase (PostgreSQL + REST APIs + Authentication)
- **Hosting:** Vercel (frontend), Firebase backend (hosted separately)
- **Browser Support:** Latest versions of Chrome, Firefox, Safari, and Edge

2.5 Design and Implementation Constraints

- The platform must operate entirely within the **free-tier limits** of Firebase and Vercel.
- All functionalities should be implementable using **open-source or no-cost libraries**.
- No AI-based suggestions, code evaluation, or hint generation is permitted.
- Only rule-based, deterministic logic should be used for problem recommendations.

2.6 User Documentation

The system will be accompanied by the following documentation components:

- An onboarding walkthrough integrated into the UI on the first login
- Contextual tooltips for all interactive elements
- A dedicated **Help** section explaining how to use each module (e.g., how to mark problems for review, interpret analytics)
- A project-level README and technical documentation in Markdown format

2.7 Assumptions and Dependencies

The following assumptions and dependencies apply to this system:

- Users have access to modern web browsers and stable internet connectivity.
- Firebase's authentication, database, and hosting services will remain available under their current free plan.
- No third-party premium APIs or commercial compilers are needed.

3. System Features

This section describes the major functional components of the PrepVault platform. Each feature is explained in terms of its purpose, behavior, and the specific requirements it fulfills. The system is organized around the core functionalities needed for effective DSA preparation: sheet management, progress tracking, problem suggestion, analytics, and revision planning. Each feature operates independently but contributes to a unified user experience aimed at structured and distraction-free learning.

3.1 User Authentication & Profile Management

Description: Allows users to register, log in, and manage profiles securely.

Priority: High

Stimulus/Response:

- Stimulus: User enters registration details (name, email, password).
- Response: System validates inputs, hashes password, and creates account.
- Stimulus: User updates profile details.
- Response: System saves changes.

Functional Requirements:

- REQ-1: The system shall allow new users to register and log in.
- REQ-2: The system shall hash all passwords before storing.
- REQ-3: The system shall allow users to update profile information.
- REQ-4: The system shall reject duplicate emails or weak passwords.

3.2 Sheet Management

Description: Enables users to create, edit, and organize custom problem sheets.

Priority: High

Stimulus/Response:

- Stimulus: User creates a new sheet (e.g., “DP Mastery”).
- Response: System stores it and displays in user dashboard.
- Stimulus: User adds a problem with metadata.
- Response: System saves problem details in the sheet.

Functional Requirements:

- REQ-5: The system shall allow creation of multiple sheets.
- REQ-6: The system shall allow adding problems with fields (title, platform, difficulty, tags, status).
- REQ-7: The system shall allow editing or deleting problems.
- REQ-8: The system shall prevent deletion of sheets containing problems unless confirmed by user.

3.3 Problem Status Tracking

Description: Allows users to track problem-solving progress with statuses.

Priority: High

Stimulus/Response:

- Stimulus: User marks a problem as Solved.
- Response: System updates sheet and progress analytics.
- Stimulus: User tags problem as Review.
- Response: Problem is added to revision planner queue.

Functional Requirements:

- REQ-9: The system shall support statuses: Not Tried, Tried, Solved, Review.
- REQ-10: The system shall update progress statistics automatically when status changes.
- REQ-11: The system shall allow filtering by tags, difficulty, and status.

3.4 Rule-Based Problem Suggestions

Description: Provides recommendations based on user's progress.

Priority: Medium

Stimulus/Response:

- Stimulus: User opens "Suggestions."
- Response: System fetches weak-topic problems or unattempted problems.

Functional Requirements:

- REQ-12: The system shall suggest problems from topics with <60% completion.
- REQ-13: The system shall avoid suggesting already solved problems.
- REQ-14: The system shall prioritize a balanced mix of easy, medium, and hard problems.

3.5 Analytics Dashboard

Description: Visual analytics of progress and strengths/weaknesses.

Priority: Medium

Stimulus/Response:

- Stimulus: User opens Analytics tab.
- Response: System displays charts (progress by topic, difficulty distribution, solved ratio).

Functional Requirements:

- REQ-15: The system shall display charts using Chart.js.
- REQ-16: The system shall show topic-wise progress percentages.
- REQ-17: The system shall show problem distribution by difficulty and tag.

3.6 Revision Planner

Description: Helps users revisit problems marked for review.

Priority: Medium

Stimulus/Response:

- Stimulus: User opens Revision Planner.
- Response: System shows list of problems with review priority.

Functional Requirements:

- REQ-18: The system shall allow users to mark problems as “Review.”
- REQ-19: The system shall prioritize revision problems based on last attempted date and weak topics.
- REQ-20: The system shall notify users of pending review problems.

4. External Interface Requirements

4.1 User Interfaces

The system will provide a **web-based graphical user interface (GUI)** accessible via modern browsers.

General Layout:

- **Navigation Bar:** Present on every page, includes Home, Sheets, Analytics, Suggestions, Revision Planner, Profile, and Logout.
- **Standard Buttons:** Add, Edit, Delete, Save, Cancel, Back, Next, Help.
- **Consistency:** Uniform style using React.js with Tailwind CSS for responsive and modern UI design.

Error Handling:

- Inline error messages displayed near invalid input fields, highlighted in red.
- Global errors (e.g., server failure) displayed in a notification banner.

Keyboard Shortcuts:

- Ctrl + N → Create new sheet.
- Ctrl + S → Save updates.
- Esc → Close modal dialogs.

Screen Constraints:

- Responsive layout for desktop, tablet, and mobile devices.
- Minimum supported resolution: **1024×768**.

4.2 Hardware Interfaces

PrepVault requires only general-purpose computing devices.

- **Supported Devices:** Desktops, laptops, tablets, and smartphones.
- **Input Devices:** Keyboard (for typing problem metadata), mouse or touch gestures (for navigation).
- **Output Devices:** Displays with minimum resolution **1024×768**.
- **No specialized hardware** is required.

4.3 Software Interfaces

4.3.1 Database Interface

- Firebase (PostgreSQL) as database.
- Stores: user accounts, sheets, problems, tags, statuses, progress logs.
- Communication: Firebase REST API and SQL queries.

4.3.2 Backend Environment

- Firebase backend with built-in authentication and database APIs.
- JSON format for request/response.

4.3.3 Frontend Environment

- React.js SPA with Tailwind CSS for styling.
- Axios/Fetch for API calls.

4.3.4 Visualization Library

- Chart.js for analytics dashboards.

4.3.5 Operating System

- Runs on cloud-hosted environment (Vercel + Firebase).
- Browser compatibility: Chrome, Firefox, Safari, Edge (latest 2 versions).

4.4 Communications Interfaces

- **Web Communication:** Frontend communicates with Firebase APIs over HTTPS.
- **Data Transfer Protocols:** HTTP/HTTPS, JSON responses.
- **Security:** TLS/SSL encryption for all client–server communications.
- **Performance:** API requests should complete within **2 seconds** under normal server load.
- **Browser Requirement:** Supports Chrome, Firefox, Safari, Edge (latest 2 versions).

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- System shall support **500 concurrent users** under free-tier hosting.
- API requests (sheet fetch, analytics load) shall respond within **300 ms**.
- Dashboard analytics updates shall compute within **1 second** for up to 5,000 problems.

5.2 Safety Requirements

- Data backup performed daily on Firebase.
- Users' problem sheets must not be lost during server outages.
- System recovery time from backup ≤ 2 hours.

5.3 Security Requirements

- All passwords hashed using bcrypt.
- All traffic secured via HTTPS/TLS 1.2+.
- Role-based access: only admin accounts can manage system-level settings (not relevant for standard users).
- User data stored following GDPR-like privacy compliance.

5.4 Software Quality Attributes

- **Availability:** 99% uptime guaranteed under free-tier hosting.
- **Maintainability:** Modular React components and Firebase schema.
- **Usability:** Mobile responsive, dark mode supported.
- **Reliability:** No data loss on browser refresh (auto-sync with backend).
- **Scalability:** Can be migrated to paid Firebase plans or containerized deployment.

6. Other Requirements

The glossary provides definitions of key terms, acronyms, and abbreviations used throughout this document. It ensures that readers can clearly understand the terminology related to PrepVault. This section helps maintain consistency and avoids ambiguity in interpretation.

6.1 Performance Requirements

- The system must handle up to **1,000 concurrent users** without significant performance degradation.
- Key operations, including problem suggestions and analytics, should respond within **3 seconds**.

6.2 Security Requirements

- User authentication must use secure methods (e.g., **JWT or OAuth 2.0**), and passwords must be encrypted both in transit and at rest.
- Access to administrative functions should be restricted to authorized personnel only.

6.3 Usability Requirements

- The user interface must be **intuitive, responsive, and mobile-friendly**.
- Navigation should allow users to perform core tasks efficiently, with tooltips and help documentation provided where necessary.

6.4 Maintainability and Extensibility

- The system code must follow standard coding conventions, enabling easy maintenance and future feature integration.
- Logging and monitoring mechanisms should be included to facilitate troubleshooting and system updates.

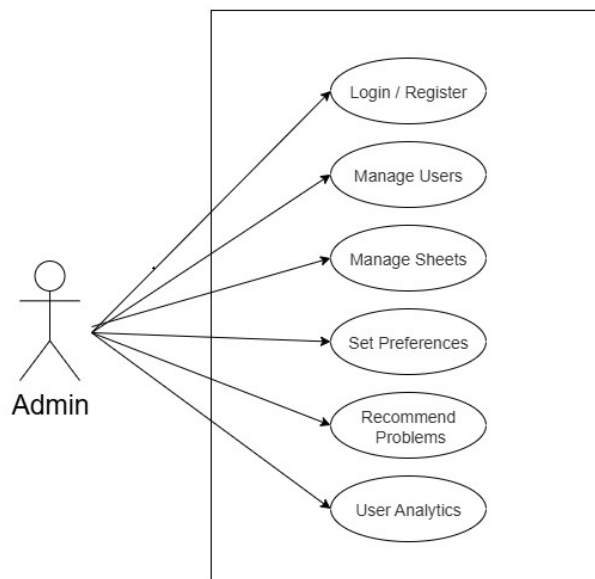
APPENDIX A : Glossary

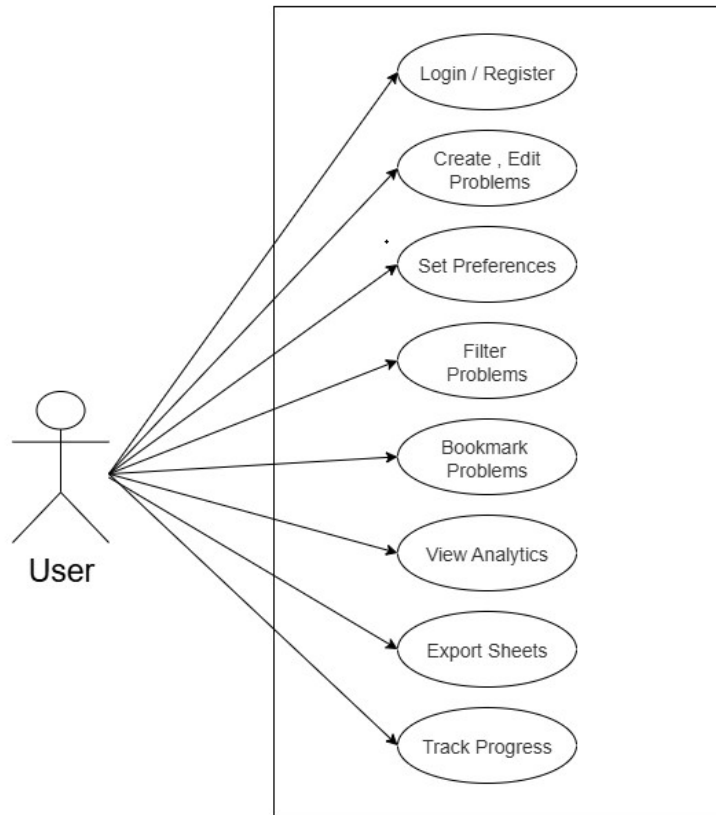
Term / Acronym	Definition
SRS	Software Requirements Specification.
DSA	Data Structures and Algorithms – the core subject area for interview preparation targeted by PrepVault.
Sheet	A user-defined collection of problems grouped for a specific goal.
Tag	A keyword or category assigned to a problem, such as <i>Dynamic Programming</i> , <i>Greedy</i> , <i>Binary Search</i> .
Difficulty	The level of problem complexity, typically categorized as <i>Easy</i> , <i>Medium</i> , <i>Hard</i> .
Rule-Based Suggestion	A deterministic recommendation system in PrepVault that suggests problems based on progress gaps (e.g., weak topics, unattempted questions).
Analytics Dashboard	A graphical visualization of progress, strengths, weaknesses, and completion statistics using charts.
Review	A special status for problems marked by the user to revisit later for revision.

APPENDIX B : Analysis Models

Use Case Diagram :

The use case diagram illustrates the interaction between users and the core functionalities of the PrepVault system. It identifies the primary actors, such as the user and the system administrator, and shows how they engage with features like user authentication, problem sheet access, progress tracking, and rule-based problem suggestions. This diagram provides a high-level view of system behavior and helps in understanding the scope of user interactions within PrepVault.





PrepVault -> Admin / User

UC1: Login / Register

- **Actor:** User, Admin
- **Precondition:** The user/admin must not be currently logged in
- **Trigger:** User/Admin selects "Login" or "Register"

Main Success Scenario:

1. User/Admin enters credentials or registration details
2. System verifies the input
3. On success, user is logged in or account is created
4. Dashboard is displayed based on role

Exception Scenarios:

- a) Invalid credentials
→ System displays error message
- b) Duplicate registration
→ System prompts that the email is already registered

- c) Network/server error
→ System shows error and allows retry

UC2: Create / Edit Problems

- **Actor:** User
- **Precondition:** User must be logged in
- **Trigger:** User selects "Create" or "Edit" from problem manager

Main Success Scenario:

1. User enters problem details
2. System validates inputs
3. Problem is saved or updated in the database
4. User is notified of success

Exception Scenarios:

- a) Missing required fields
→ System prompts user to complete form
- b) Duplicate title
→ System asks to use a unique title

UC3: Filter Problems

- **Actor:** User
- **Precondition:** User must be logged in
- **Trigger:** User selects filter options (e.g., topic, difficulty)

Main Success Scenario:

1. User applies filters
2. System displays matching problems
3. User can take further actions (solve, bookmark)

Exception Scenarios:

- a) No matching results
→ System shows "No problems found" message

UC4: Set Preferences

- **Actor:** User, Admin
- **Precondition:** Must be logged in
- **Trigger:** User/Admin accesses "Preferences" section

Main Success Scenario:

1. User/Admin sets or updates preferences (e.g., topic focus, difficulty)
2. System saves and applies preferences
3. Problem recommendations update accordingly

Exception Scenarios:

- a) Invalid values or missing fields
→ System shows validation messages

UC5: Bookmark Problems

- **Actor:** User
- **Precondition:** User is logged in and viewing a problem
- **Trigger:** User selects "Bookmark"

Main Success Scenario:

1. System adds problem to user's bookmarks
2. Bookmark list is updated

Exception Scenarios:

- a) Already bookmarked
→ System shows duplicate notice or disables the button

UC6: View Analytics

- **Actor:** User

- **Precondition:** User must have solved some problems
- **Trigger:** User selects "Analytics"

Main Success Scenario:

1. System calculates and displays progress stats
2. User views strengths, weaknesses, and topic mastery

Exception Scenarios:

- a) No activity data
→ System informs "No activity found"

UC7: Export Sheets

- **Actor:** User
- **Precondition:** User is logged in
- **Trigger:** User clicks "Export Sheets"

Main Success Scenario:

1. User selects desired sheet
2. System generates downloadable PDF/CSV
3. File is downloaded

Exception Scenarios:

- a) Export fails due to file generation error
→ System displays error and retry option

UC8: Track Progress

- **Actor:** User
- **Precondition:** User has attempted problems
- **Trigger:** User selects "Track Progress"

Main Success Scenario:

1. System retrieves solved/unsolved problem history
2. Progress chart is shown
3. Suggestions are optionally displayed

UC9: Manage Users

- **Actor:** Admin
- **Precondition:** Admin must be logged in
- **Trigger:** Admin selects "Manage Users"

Main Success Scenario:

1. Admin views list of registered users
2. Admin can activate, deactivate, or remove users
3. Changes are saved and reflected immediately

Exception Scenarios:

- a) Unauthorized action
→ System blocks and notifies

UC10: Manage Sheets

- **Actor:** Admin
- **Precondition:** Admin is logged in
- **Trigger:** Admin selects "Manage Sheets"

Main Success Scenario:

1. Admin creates or edits sheets (topic-wise sets)
2. System saves and syncs for users
3. Users receive updated content

UC11: Recommend Problems

- **Actor:** Admin
- **Precondition:** Admin is logged in

- **Trigger:** Admin selects "Recommend Problems"

Main Success Scenario:

1. Admin views user analytics
2. Admin recommends personalized problems
3. Users see new recommended tasks

UC12: View User Analytics

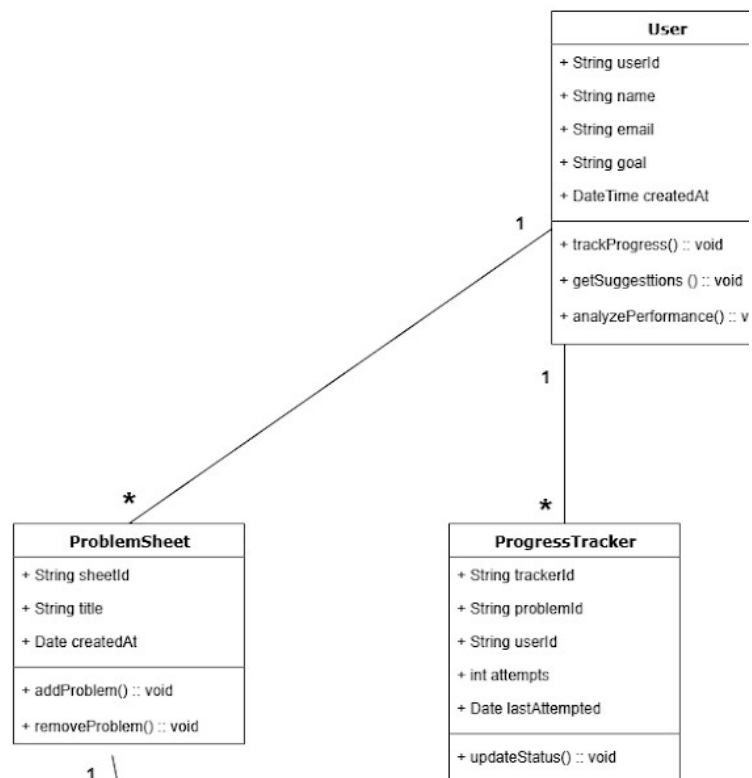
- **Actor:** Admin
- **Precondition:** Admin is logged in
- **Trigger:** Admin selects "User Analytics"

Main Success Scenario:

1. Admin accesses analytics dashboard
2. Progress, login activity, and topic coverage per user is shown
3. Admin uses insights to support users

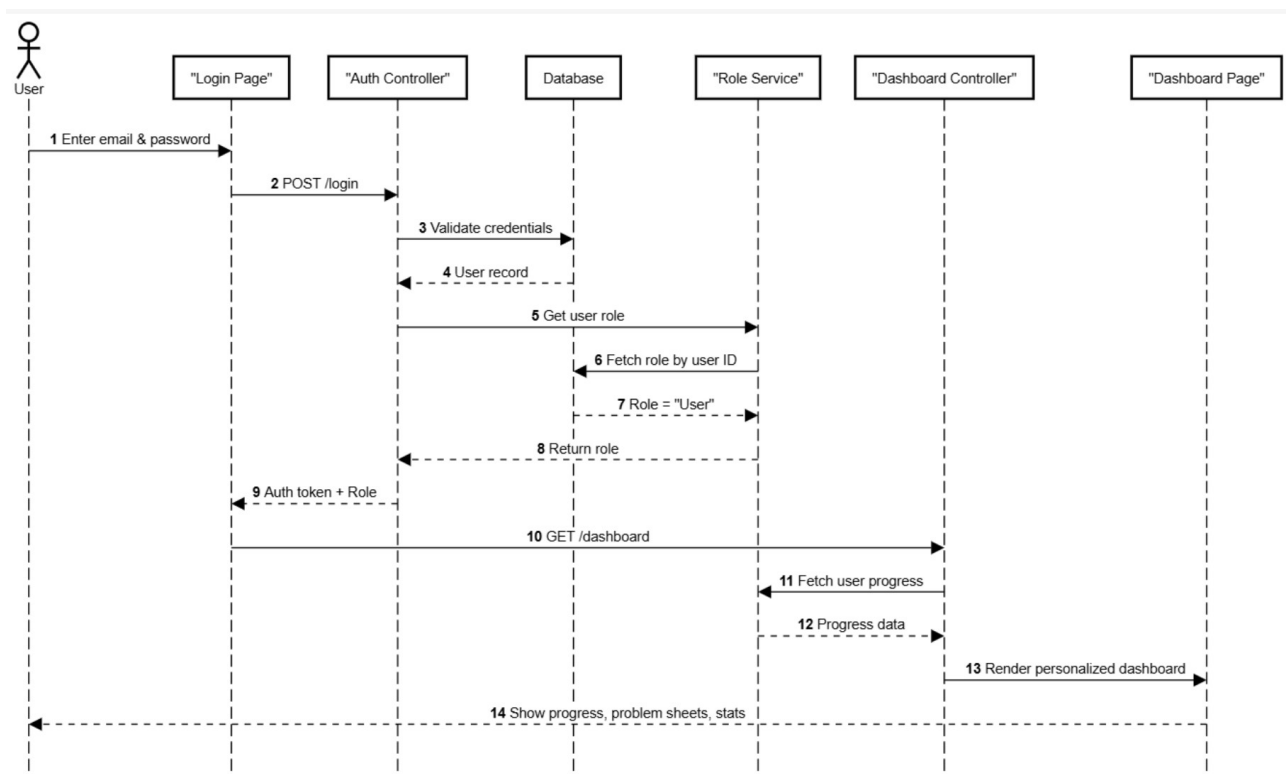
Class Diagram :

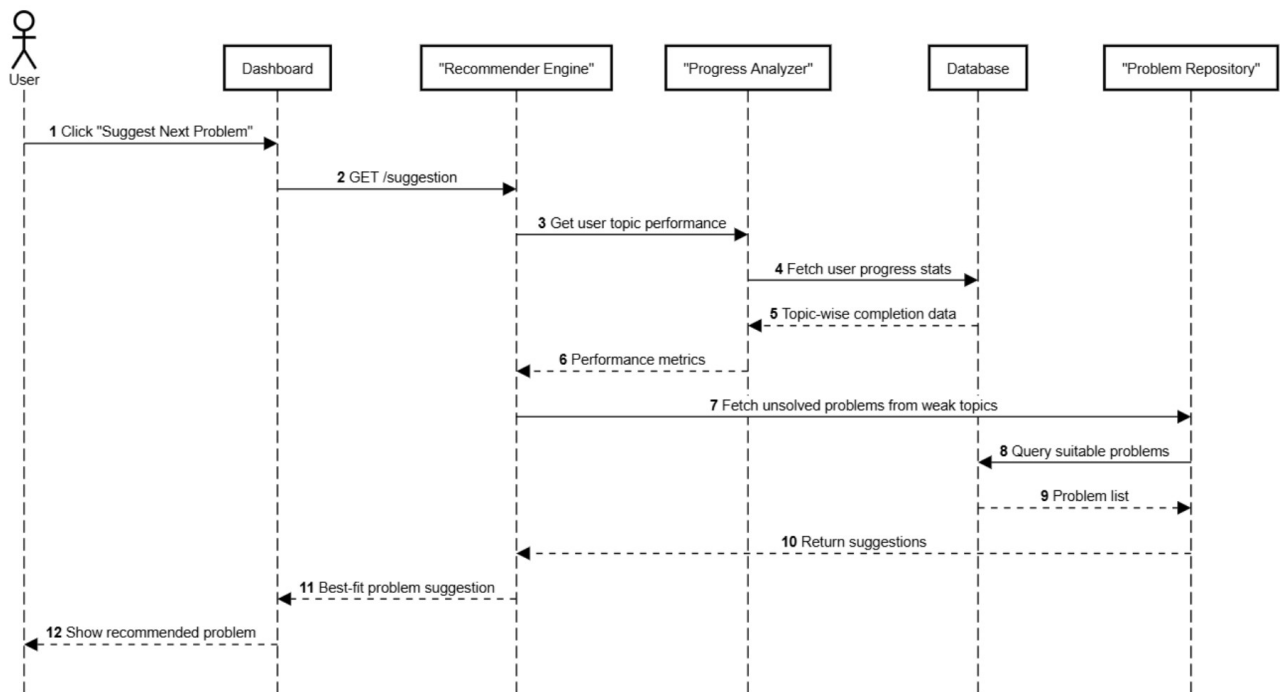
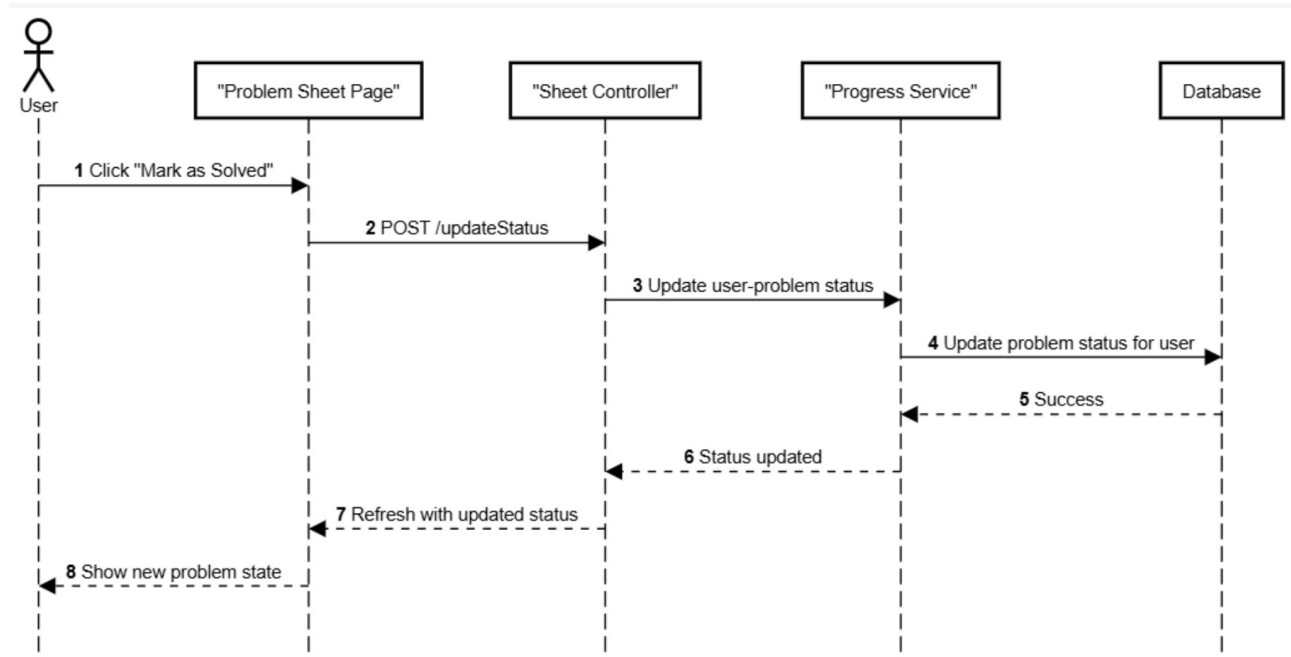
The class diagram represents the static structure of the PrepVault system by defining its main classes, their attributes, and the relationships among them. It provides a blueprint of how different components, such as User, ProblemSheet, Problem, ProgressTracker, and SuggestionEngine interact with each other. This diagram helps in understanding the object-oriented design of the system and serves as a foundation for the implementation phase.

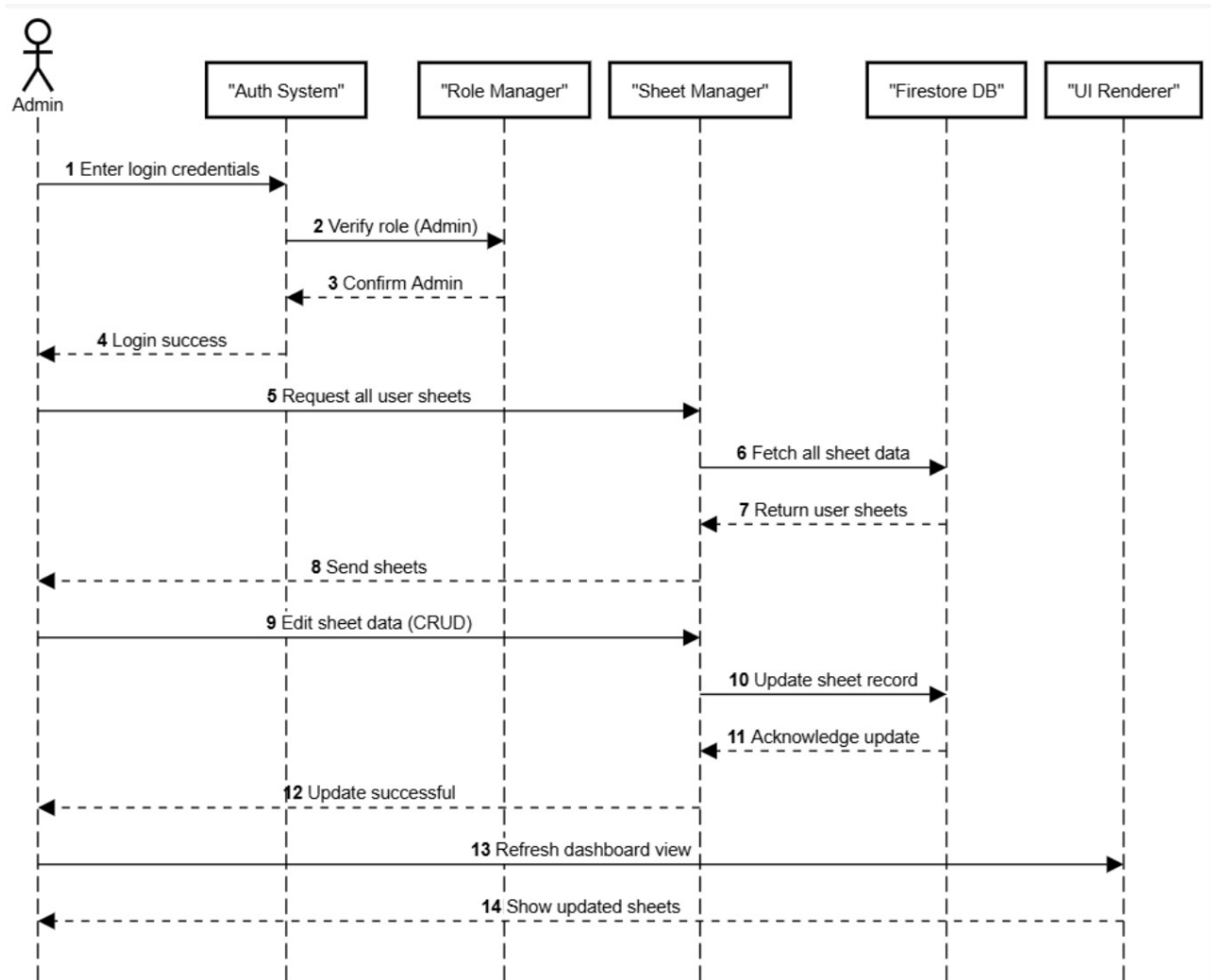


Sequence Diagram :

The sequence diagram depicts the flow of interactions between different system components in a time-ordered manner. It shows how objects such as the User, Authentication Module, Problem Sheet Manager, Progress Tracker, and Suggestion Engine communicate with one another to complete specific tasks, such as logging in, viewing problem sheets, or receiving problem suggestions. This diagram helps in visualizing the dynamic behavior of the PrepVault system and clarifies the sequence of operations required to achieve system functionality.

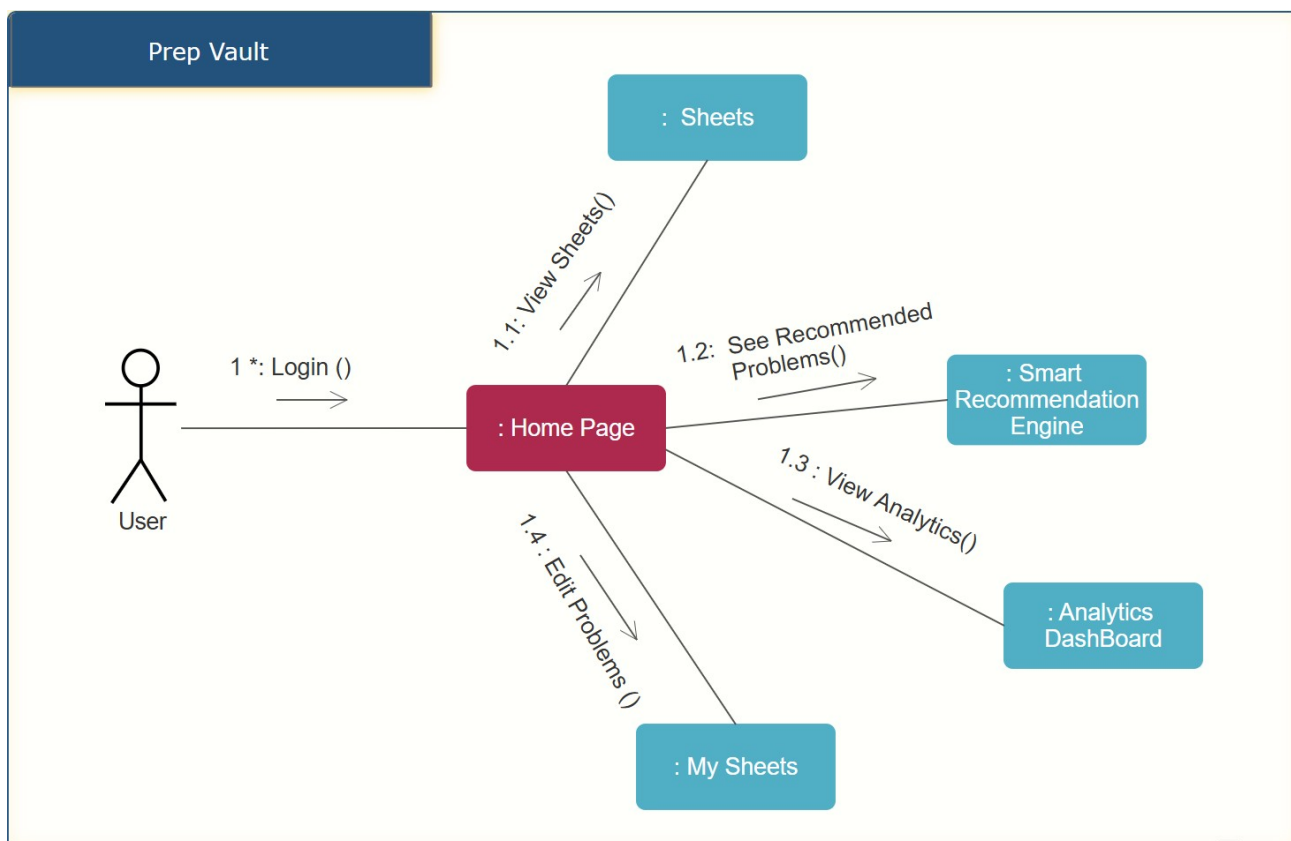






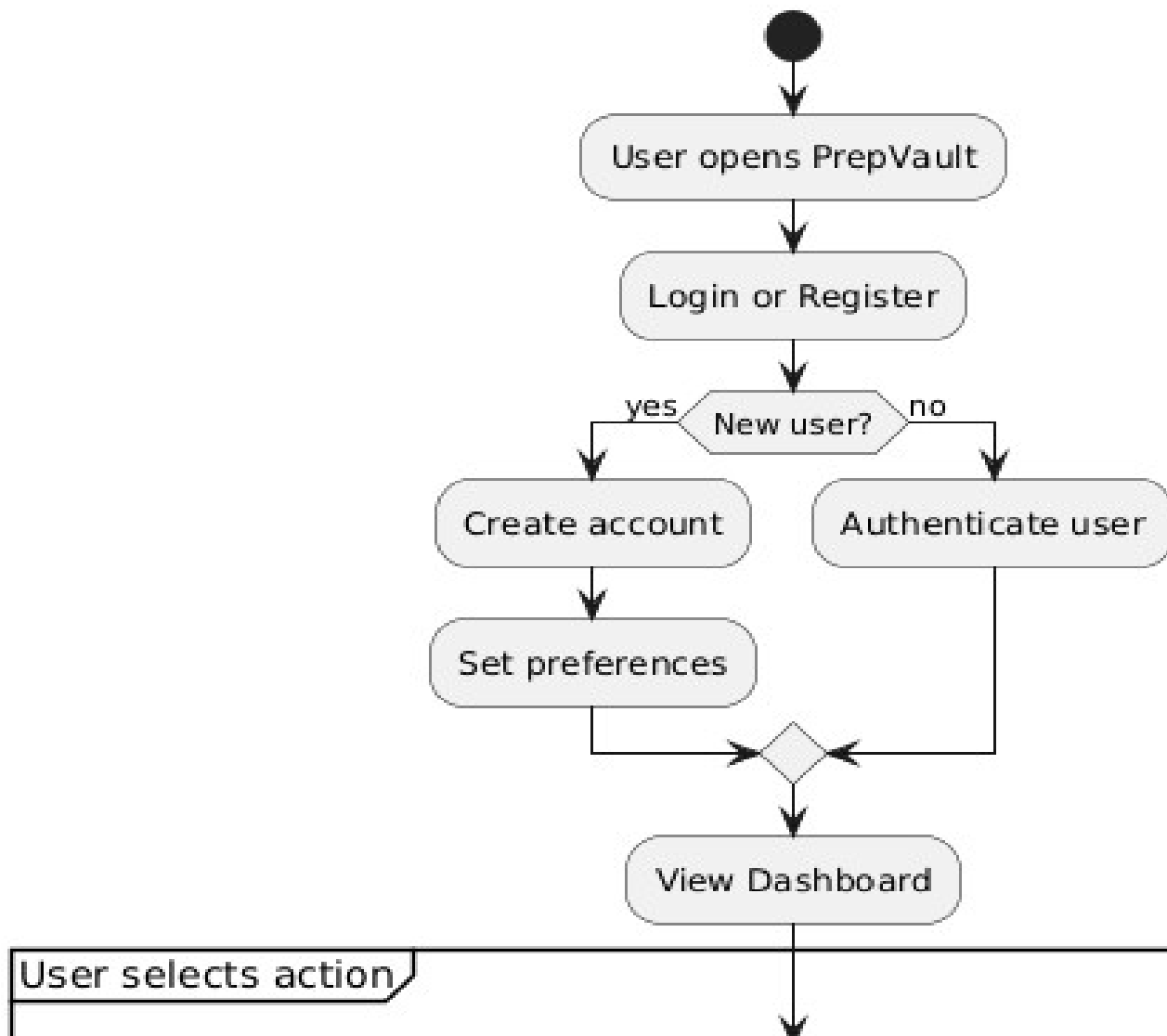
Colaboration Diagram :

The collaboration diagram illustrates the structural organization of the PrepVault system by showing the relationships and interactions between objects. It emphasizes how objects such as the User, Authentication Module, Problem Sheet Manager, and Progress Tracker collaborate with one another to accomplish tasks. Unlike the sequence diagram, which focuses on the time order of messages, the collaboration diagram highlights the roles and links between components, providing a clearer view of system communication patterns.

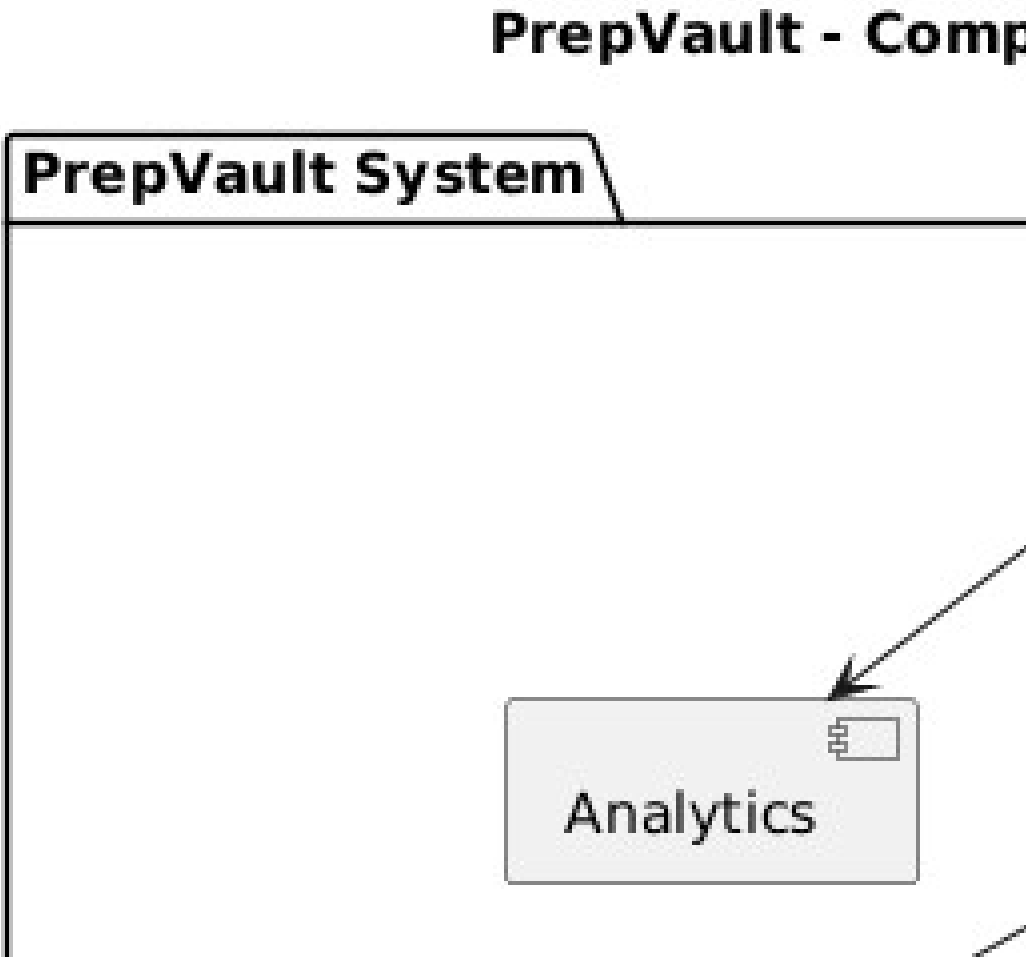


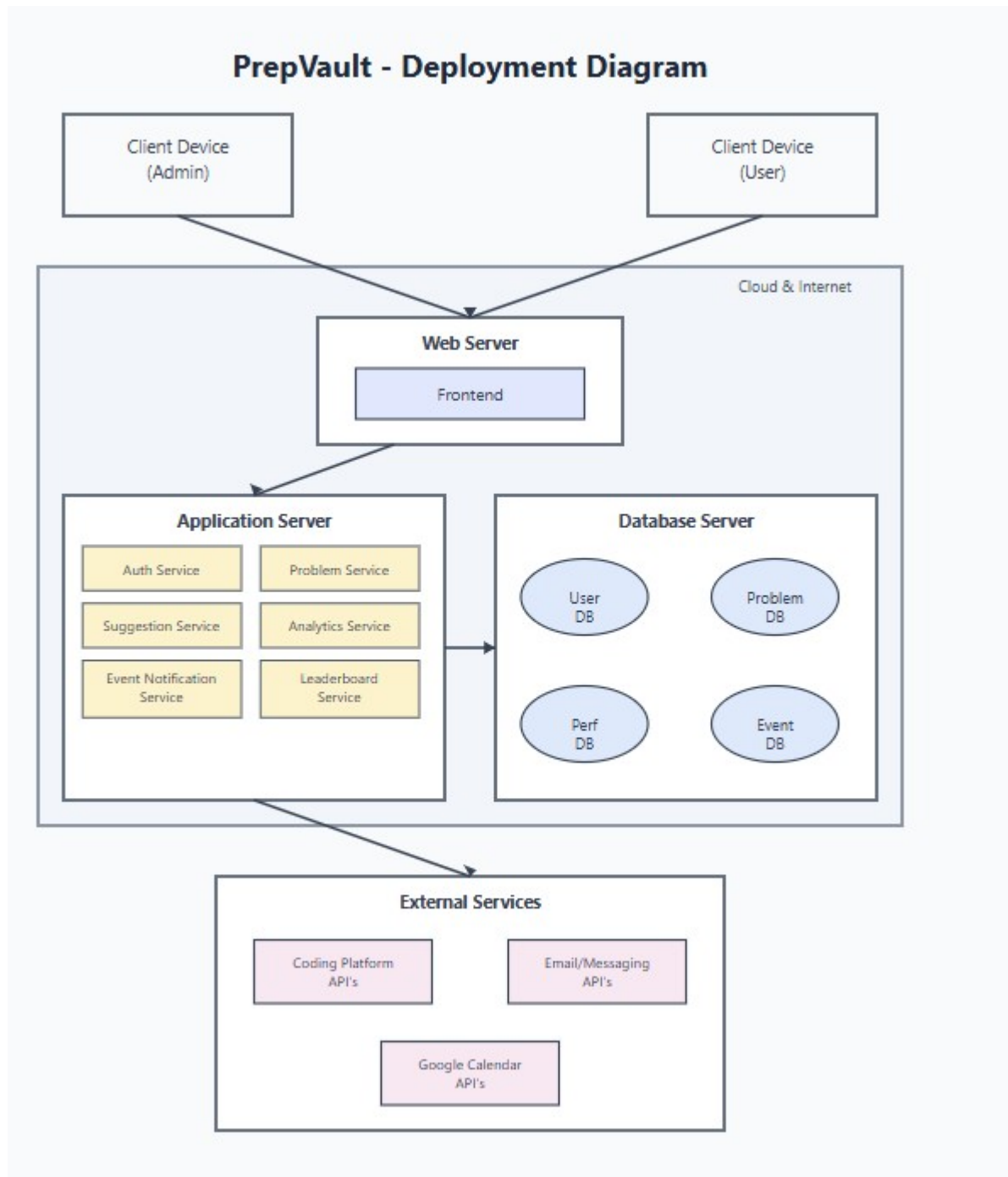
Activity Diagram :

The activity diagram represents the workflow of the PrepVault system by modeling the sequence of activities and decision points involved in completing a task. It captures processes such as user login, accessing problem sheets, solving problems, updating progress, and receiving suggestions. This diagram provides a clear view of the dynamic behavior of the system, making it easier to understand the flow of operations and the possible paths a user can take within PrepVault.



Component Diagram :



Deployment Diagram :

APPENDIX C : Source Code

App.js

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Layout from '../components/Layout';
import AdminLayout from '../components/AdminLayout';
import Home from './Pages/Home';
import Sheets from './Pages/Sheets';
import SheetDetail from './Pages/SheetDetail';
import Analytics from './Pages/Analytics';
import Problems from './Pages/Problems';
import LoginForm from './Pages/LoginForm';
import RegisterForm from './Pages/RegisterForm';
import Profile from './Pages/Profile';
import AdminLoginForm from './Pages/AdminLoginForm';
import AdminAnalytics from './Pages/AdminAnalytics';
import UserManagement from './Pages/UserManagement';
import AdminDebug from './Pages/AdminDebug';
import PrivateRoute from '../components/PrivateRoute';
import AdminRoute from '../components/AdminRoute';
import { AuthProvider } from './context/AuthContext';
import ProblemDetail from './Pages/ProblemDetail';
```

```
function App() {
  return (
    <AuthProvider>
      <Router>
        <Routes>
          { /* Regular User Routes */ }
          <Route path="/" element={ <LoginForm /> } />
          <Route path="/register" element={ <RegisterForm /> } />
          <Route
            path="/home"
            element={
              <PrivateRoute>
                <Layout>
                  <Home />
                </Layout>
              </PrivateRoute>
            }
          />
          <Route
            path="/sheets"
            element={
              <PrivateRoute>
```

```
        <Layout>
          <Sheets />
        </Layout>
      </PrivateRoute>
    }
  />
  <Route
    path="/sheets/:sheetId"
    element={
      <PrivateRoute>
        <Layout>
          <SheetDetail />
        </Layout>
      </PrivateRoute>
    }
  />
  <Route
    path="/problems/:problemId"
    element={
      <PrivateRoute>
        <Layout>
          <ProblemDetail />
        </Layout>
      </PrivateRoute>
    }
  />
  <Route
    path="/analytics"
    element={
      <PrivateRoute>
        <Layout>
          <Analytics />
        </Layout>
      </PrivateRoute>
    }
  />
  <Route
    path="/profile"
    element={
      <PrivateRoute>
        <Layout>
          <Profile />
        </Layout>
      </PrivateRoute>
    }
  />
```

```

    { /* Admin Routes */}
    <Route path="/admin/login" element={<AdminLoginForm />} />
    <Route path="/admin/debug" element={<AdminDebug />} />
    <Route
      path="/admin/analytics"
      element={
        <AdminRoute>
          <AdminLayout>
            <AdminAnalytics />
          </AdminLayout>
        </AdminRoute>
      }
    />
    <Route
      path="/admin/users"
      element={
        <AdminRoute>
          <AdminLayout>
            <UserManagement />
          </AdminLayout>
        </AdminRoute>
      }
    />
  </Routes>
</Router>
</AuthProvider>
);
}

```

```
export default App;
```

SheetCard.jsx

```

// src/components/SheetCard.jsx
import React from "react";
import { Link } from "react-router-dom";
import { Card, CardContent } from "../ui/Card";
import { Button } from "../ui/Button";
import styles from "../SheetCard.module.css";

export default function SheetCard({ sheet, problemCount, solvedCount, onDelete }) {
  const progress = problemCount > 0 ? Math.round((solvedCount / problemCount) * 100) : 0;

  return (
    <Card className={styles.sheetCard}>
      { /* Progress Bar */}
    </Card>
  );
}

```

```

<div className={` ${styles.progressBar} ${progress === 100 ? styles.fullProgress : ""} `}
style={{ width: `${progress}%` }}>
  <span className={styles.progressBarText}>{progress}%</span>
</div>

<CardContent>
  <div className={styles.sheetHeader}>
    <h3 className={styles.sheetTitle}>{sheet.name}</h3>
    <p className={styles.sheetDescription}>{sheet.description || "\u00A0"}</p> { /* Non-
breaking space for empty description */}
  </div>

  <div className={styles.sheetStats}>
    <div className={styles.statItem}>
      <span className={styles.statIcon}>≡</span>
      <span>{problemCount} Questions</span>
    </div>
    <div className={styles.statItem}>
      <span className={styles.statIcon}>✓</span>
      <span className={styles.solvedCount}>{solvedCount} Solved</span>
    </div>
    <div className={styles.statItem}>
      <span className={styles.statIcon}><img alt="calendar icon" data-bbox="458 425 478 440"/></span>
      <span>{new Date(sheet.createdAt).toLocaleDateString()}</span>
    </div>
  </div>

  <div className={styles.sheetActions}>
    <Link to={` /sheets/${sheet.id} `} className={styles.viewButton}>
      View Problems
    </Link>
    <Button
      variant="outline"
      size="sm"
      onClick={() => onDelete(sheet.id)}
      className={styles.deleteButton}
    >
      Delete
    </Button>
  </div>
</CardContent>
</Card>
);
}

```

LoginForm.jsx

```

import React, { useState } from "react";
import { signInWithEmailAndPassword } from "firebase/auth";
import { auth } from "../firebase";
import { useNavigate, Link } from "react-router-dom";
import styles from "../Auth.module.css";

export default function LoginForm() {

```

```

const [formData, setFormData] = useState({ Email: "", Password: "" });
const navigate = useNavigate();

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData((prev) => ({ ...prev, [name]: value }));
};

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    const userCredential = await signInWithEmailAndPassword(
      auth,
      formData.Email,
      formData.Password
    );
    const user = userCredential.user;
    alert(`Welcome ${user.email}!`);
    navigate("/home");
  } catch (error) {
    alert(`Login failed: ${error.message}`);
  }
};

return (
  <div className={styles.authContainer}>
    <form onSubmit={handleSubmit} className={styles.authForm}>
      <h2 className={styles.authHeading}>PrepVault Login</h2>

      <label className={styles.authLabel}>Email:</label>
      <input
        type="email"
        name="Email"
        value={formData.Email}
        onChange={handleChange}
        required
        className={styles.authInput}
      />

      <label className={styles.authLabel}>Password:</label>
      <input
        type="password"
        name="Password"
        value={formData.Password}
        onChange={handleChange}
        required
        className={styles.authInput}
      />

      <button type="submit" className={styles.authButton}>Login</button>

      <p className={styles.authLink}>
        Don't have an account? <Link to="/register">Register</Link>
      </p>
    </form>
  </div>
);

```



```

    <p className={styles.authLink} style={{ marginTop: '10px', fontSize: '14px' }}>
      Are you an admin? <Link to="/admin/login">Admin Login</Link>
    </p>
  </form>
</div>
);
}

```

Analytics.jsx

```

import React, { useEffect, useState } from "react";
import { db, auth } from "../firebase";
import { collection, getDocs, query, where, doc, getDoc } from "firebase/firestore";
import { Card, CardContent } from "../../components/ui/Card";
import { PieChart, Pie, Cell, Tooltip, ResponsiveContainer } from "recharts";
import styles from "./Analytics.module.css";

export default function Analytics() {
  const [stats, setStats] = useState({ solved: 0, attempted: 0, unsolved: 0 });
  const [userName, setUserName] = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      // Check if user is authenticated
      if (!auth.currentUser) {
        console.warn("No authenticated user, cannot fetch analytics");
        setLoading(false);
        return;
      }

      try {
        setLoading(true);
        setError(null);

        // Fetch user profile first
        const userDocRef = doc(db, "users", auth.currentUser.uid);
        const userDoc = await getDoc(userDocRef);
        if (userDoc.exists()) {
          const userData = userDoc.data();
          setUserName(userData.name || auth.currentUser.email.split('@')[0]);
        } else {
          setUserName(auth.currentUser.email.split('@')[0]);
        }

        const problemsRef = collection(db, "problems");
        // Query only problems belonging to the current user
        const q = query(problemsRef, where("user", "=", auth.currentUser.uid));
        const snapshot = await getDocs(q);

        let solved = 0, attempted = 0, unsolved = 0;

```

```

    snapshot.docs.forEach((doc) => {
      const data = doc.data();
      // Status validation
      if (data.status === "Solved") solved++;
      else if (data.status === "Attempted") attempted++;
      else if (data.status === "Unsolved") unsolved++;
      // Handle any other status as "Unsolved" for safety
    });

    setStats({ solved, attempted, unsolved });
  } catch (err) {
    console.error("Error fetching analytics data:", err);
    setError("Failed to load analytics data");
  } finally {
    setLoading(false);
  }
};

fetchData();
}, []);

const chartData = [
  { name: "Solved", value: stats.solved },
  { name: "Attempted", value: stats.attempted },
  { name: "Unsolved", value: stats.unsolved },
];

const COLORS = ["#22c55e", "#facc15", "#ef4444"];

// Handle loading and error states
if (loading) {
  return (
    <div className={styles.analytics}>
      <h2 className={styles.pageTitle}>Analytics  </h2>
      <p>Loading analytics data...</p>
    </div>
  );
}

if (error) {
  return (
    <div className={styles.analytics}>
      <h2 className={styles.pageTitle}>Analytics  </h2>
      <p style={{ color: 'red' }}>{error}</p>
    </div>
  );
}

return (
  <div className={styles.analytics}>
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: '30px' }}>
      <h2 className={styles.pageTitle}>Analytics  {userName && ` - ${userName}`} </h2>

```

```

    {userName && !userName.includes('@') && userName !==
auth.currentUser?.email?.split('@')[0] && (
    <p style={{ color: '#6b7280', fontSize: '14px' }}>
      Welcome back, {userName}! 🙌
    </p>
  )}
  {(!userName || userName === auth.currentUser?.email?.split('@')[0]) && (
    <p style={{ color: '#6b7280', fontSize: '14px' }}>
      Complete your <a href="/profile" style={{ color: '#3b82f6', textDecoration: 'none'
}}>profile</a> to show your name on leaderboards
    </p>
  )}
</div>

<div className={styles.statsGrid}>
  <Card className={` ${styles.statCard} ${styles.solved} `}>
    <CardContent>
      <p className={` ${styles.statValue} ${styles.solved} `}>{stats.solved}</p>
      <p className={styles.statLabel}>Solved</p>
    </CardContent>
  </Card>
  <Card className={` ${styles.statCard} ${styles.attempted} `}>
    <CardContent>
      <p className={` ${styles.statValue} ${styles.attempted} `}>{stats.attempted}</p>
      <p className={styles.statLabel}>Attempted</p>
    </CardContent>
  </Card>
  <Card className={` ${styles.statCard} ${styles.unsigned} `}>
    <CardContent>
      <p className={` ${styles.statValue} ${styles.unsigned} `}>{stats.unsigned}</p>
      <p className={styles.statLabel}>Unsigned</p>
    </CardContent>
  </Card>
</div>

<div className={styles.chartContainer}>
  <ResponsiveContainer width="100%" height="100%">
    <PieChart>
      <Pie
        data={chartData}
        dataKey="value"
        outerRadius={100}
        label
      >
        {chartData.map((entry, index) => (
          <Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]} />
        ))}
      </Pie>
      <Tooltip />
    </PieChart>
  </ResponsiveContainer>
</div>
</div>
);}

```

APPENDIX D : Output Screenshots

USER LOGIN :

My Profile

Basic Information

Full Name *

Praveenkumar

This name will be displayed on the leaderboard

Email

praveenredmi7@gmail.com

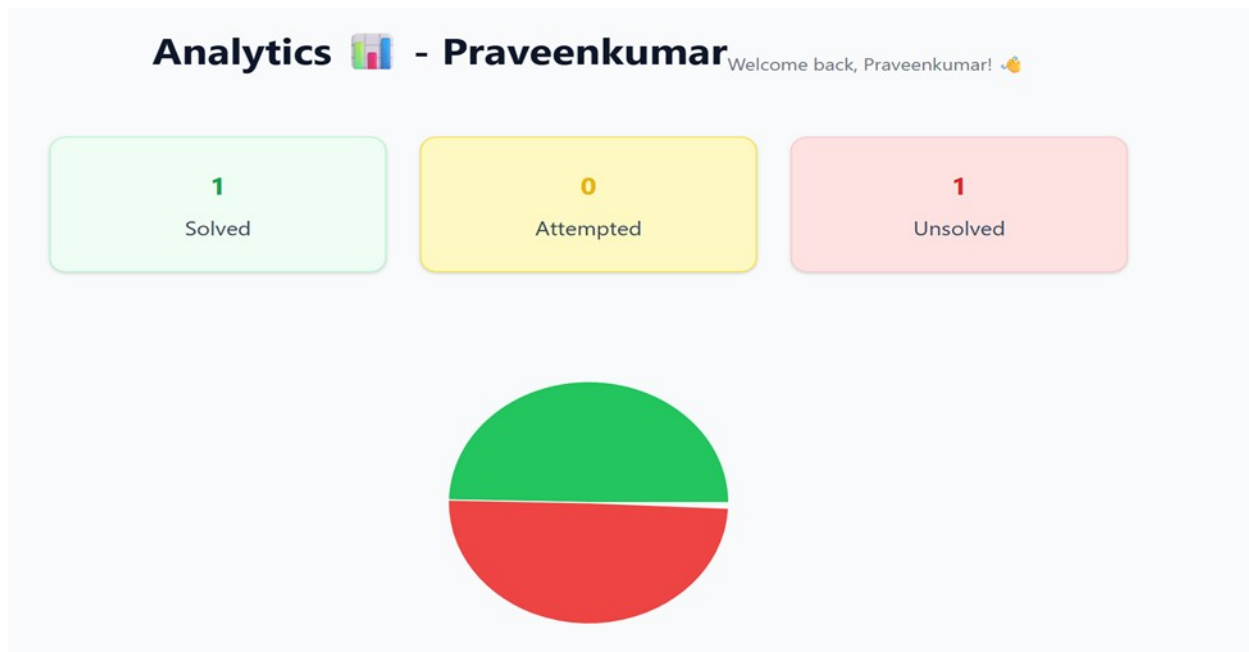
Email cannot be changed

Contact Number

+916369286869

Professional Information

USER REGISTRATION :



admin@prepvault.com

Admin Password:

Admin Login

Regular user? [User Login](#)

235

PrepVault Admin Login

Administrator Access Only

Admin Email:

admin@prepvault.com

Admin Password:

Admin Login

Regular user? [User Login](#)

USER HOMEPAGE :

PrepVault Sheets Analytics Profile Logout

PrepVault
Your DSA practice dashboard

Open Sheets

Welcome back, **Praveenkumar!** 🙌 Ready to solve some problems?

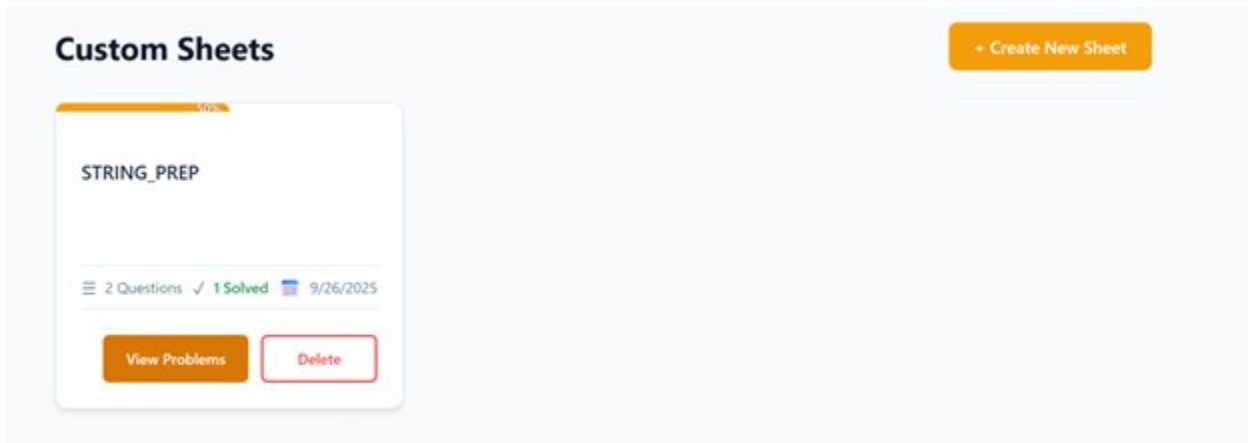
Solved 1	Attempted 0	Unsolved 1
--------------------	-----------------------	----------------------

Recent Problems [See all](#)

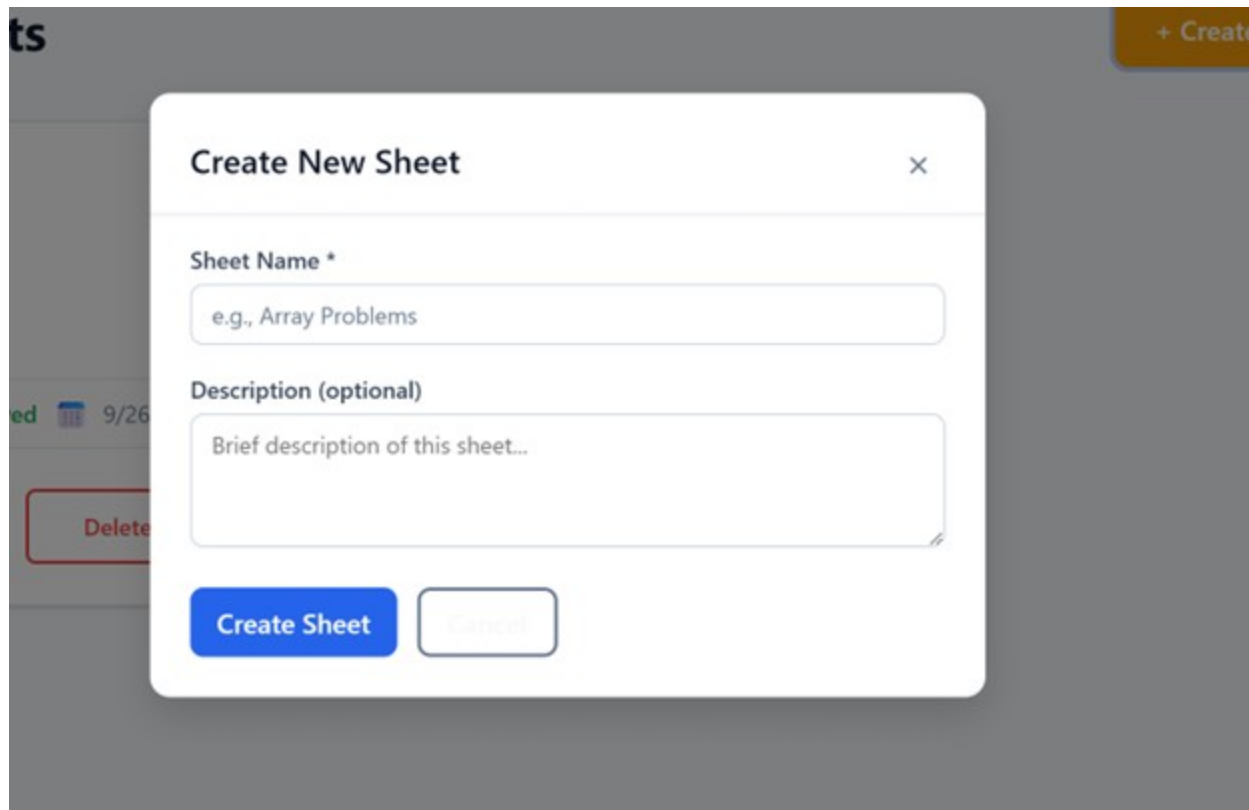
Aggressive Cows Sorting, Two Pointers	Unsolved Edit
Three Sum	Solved Edit

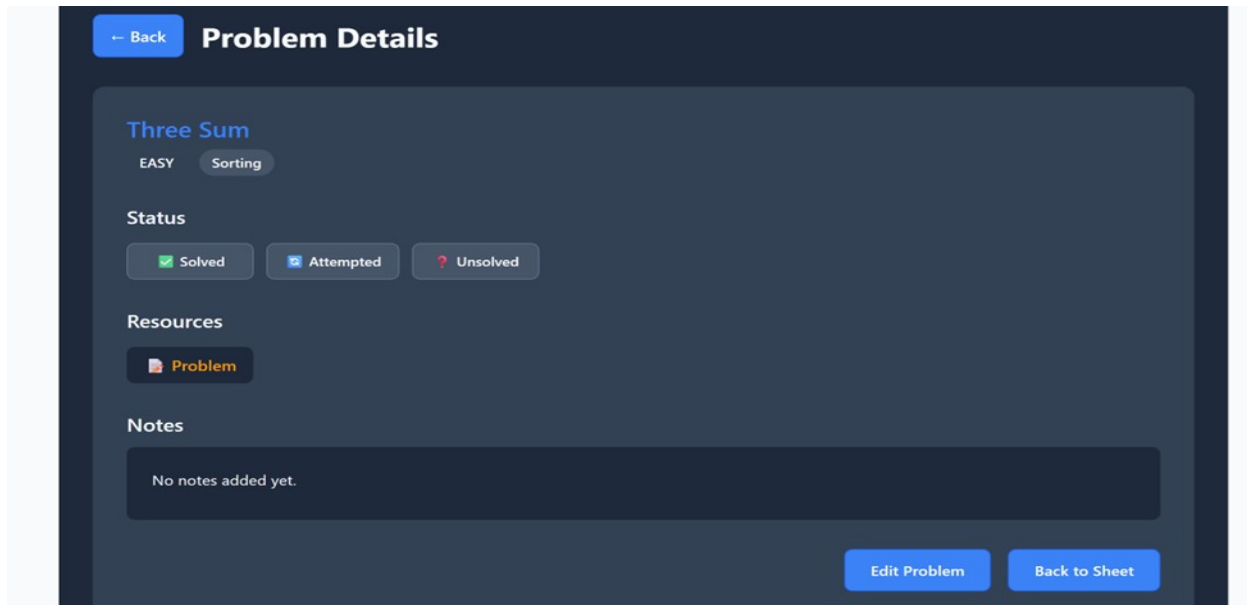
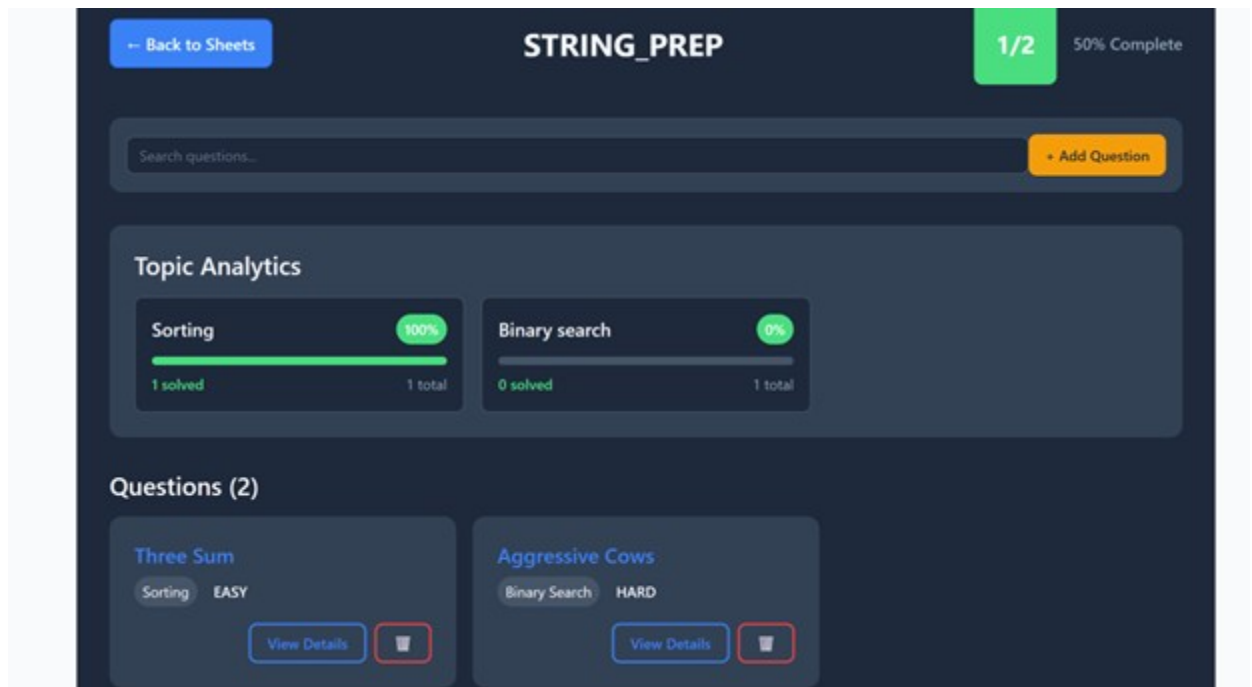
Quick Add
Add a problem quickly from here.
[Add Problem](#)

MY SHEETS :



CREATE SHEET :



PROBLEM DETAILS :**SHEET DETAILS :**

USER PROFILE :

My Profile

Basic Information

Full Name *

Praveenkumar

This name will be displayed on the leaderboard

Email

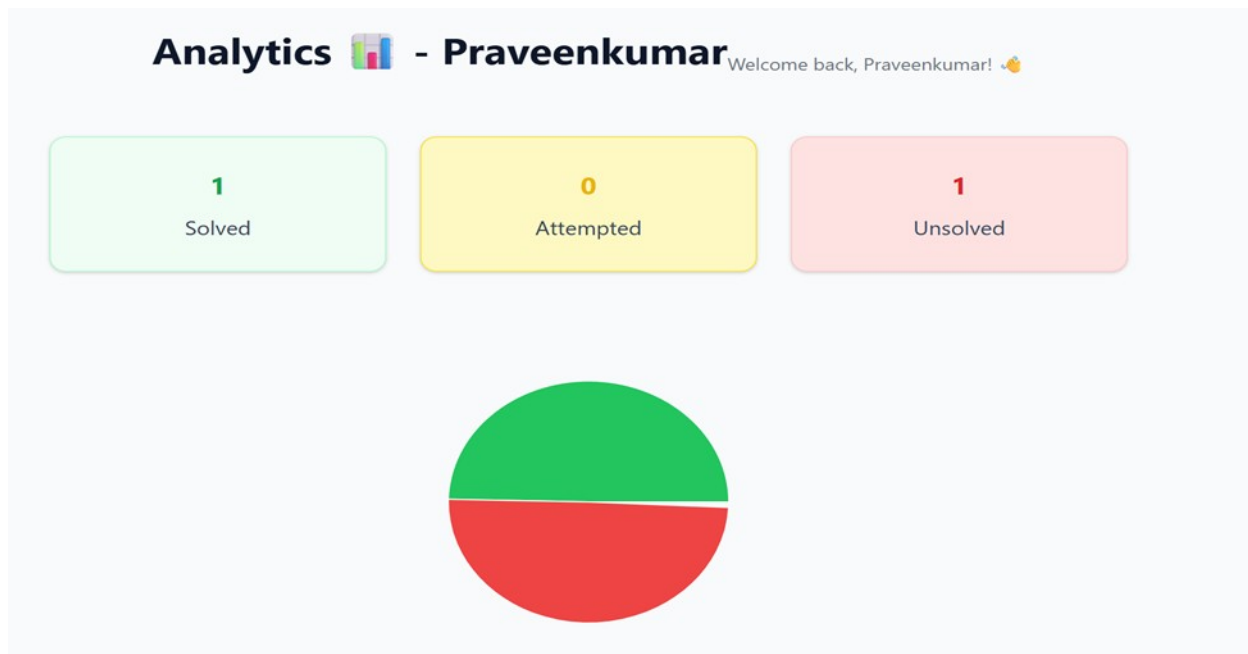
praveenredmi7@gmail.com

Email cannot be changed

Contact Number

+916369285869

Professional Information

USER ANALYTICS :

ADMIN DASHBOARD 1 :**ADMIN DASHBOARD 2 :**